

**UNIVERSIDAD PRIVADA DE TACNA**

**FACULTAD DE INGENIERÍA**

**ESCUELA DE INGENIERÍA DE SISTEMAS**



# **Guía Práctica de Laboratorio**

**“ALINEACIÓN DE FÚTBOL”**

**“PROGRAMACIÓN II”**

**DOCENTE:**

**Ing. Breno Luque Zuñiga**

**ESTUDIANTE:**

**Arturo Sebastian Cutipa Flores**

**TACNA – PERÚ**

**2026**



|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Guía de Laboratorio N° 02 Unidad 2 “MÁQUINA EXPENDEDORA# ”</b> | <b>4</b>  |
| <b>I. INFORMACIÓN GENERAL</b>                                     | <b>4</b>  |
| Objetivos                                                         | 4         |
| Equipos, materiales, programas y recursos utilizados              | 4         |
| <b>II. MARCO TEÓRICO</b>                                          | <b>4</b>  |
| Windows Forms                                                     | 4         |
| Programación Orientada a Objetos                                  | 4         |
| DataGridView                                                      | 4         |
| <b>III. PROCEDIMIENTO</b>                                         | <b>5</b>  |
| Paso 1: Crear el proyecto                                         | 5         |
| Paso 2: Diseñar la interfaz gráfica                               | 5         |
| Paso 3: Creación de la clase Jugador                              | 6         |
| Paso 4: Declaración de listas                                     | 6         |
| Paso 5: Implementación del método Mostrar()                       | 7         |
| Paso 6: Programación del botón Agregar                            | 7         |
| Paso 7: Programación del botón Eliminar                           | 8         |
| Paso 8: Programación del botón Modificar                          | 8         |
| Paso 9: Programación del DataGridView                             | 8         |
| Paso 10: Ejecución y pruebas                                      | 9         |
| <b>IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS</b>                | <b>10</b> |
| Interfaz principal del sistema:                                   | 10        |
| Registro de jugadores:                                            | 11        |
| Modificación de jugadores:                                        | 12        |
| Eliminación de jugadores:                                         | 13        |
| Funcionamiento general del sistema:                               | 13        |
| <b>V. CUESTIONARIO</b>                                            | <b>14</b> |
| 1. ¿Qué es Windows Forms?                                         | 14        |
| 2. ¿Qué es Programación Orientada a Objetos?                      | 14        |
| 3. ¿Qué es una clase?                                             | 14        |
| 4. ¿Qué es List?                                                  | 14        |
| 5. ¿Qué función cumple el DataGridView?                           | 14        |
| <b>CONCLUSIONES</b>                                               | <b>14</b> |
| <b>RECOMENDACIONES</b>                                            | <b>14</b> |
| <b>BIBLIOGRAFÍA</b>                                               | <b>14</b> |
| <b>WEBGRAFÍA</b>                                                  | <b>15</b> |



## **Guía de Laboratorio N° 02 Unidad 2 “ALINEACIÓN DE FÚTBOL”**

### **I. INFORMACIÓN GENERAL**

#### **Objetivos**

- Desarrollar una aplicación de escritorio utilizando Windows Forms.
- Aplicar Programación Orientada a Objetos (POO).
- Implementar operaciones CRUD en memoria.
- Utilizar DataGridView para visualizar información.
- Manipular listas genéricas en C#.

#### **Equipos, materiales, programas y recursos utilizados**

- Programación Orientada a Objetos.
- Colecciones List.
- DataGridView.

### **II. MARCO TEÓRICO**

#### **Windows Forms**

Windows Forms es una tecnología de Microsoft utilizada para desarrollar aplicaciones de escritorio con interfaz gráfica en C#. Permite trabajar con controles visuales como botones, etiquetas, cuadros de texto y tablas.

#### **Programación Orientada a Objetos**

La Programación Orientada a Objetos (POO) es un paradigma de programación basado en clases y objetos. Permite organizar mejor la información y reutilizar código.

Los conceptos utilizados fueron:

- Clases
- Objetos
- Encapsulamiento
- Métodos
- Colecciones

#### **DataGridView**

El DataGridView es un control de Windows Forms que permite mostrar información en forma de tabla.

Fue utilizado para visualizar los jugadores registrados.

### **III. PROCEDIMIENTO**

#### **Paso 1: Crear el proyecto**

Se abrió el programa Visual Studio y se creó un nuevo proyecto de tipo Windows Forms App (.NET Framework) con el nombre: Alinacionfutbol

Después de crear el proyecto, se ingresó al formulario principal (Form1) donde se desarrolló toda la aplicación.

#### **Paso 2: Diseñar la interfaz gráfica**

Se agregaron los siguientes controles:

##### **Labels**

- Nombre
- Posición
- Número
- Condición

##### **TextBox**

- txtNombre
- txtPosicion
- txtNumero

##### **ComboBox**

- cbCondicion

##### **Opciones:**

- Titular
- Suplente

##### **Buttons**

- btnAgregar
- btnModificar
- btnEliminar

##### **DataGridView**

- dataGridView1



### Paso 3: Creación de la clase Jugador

Se creó una clase llamada Jugador.cs para almacenar la información de los productos disponibles.

```
CSharp
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace AlineacionFutbol
{
    internal class Jugador
    {
        public string Nombre { get; set; }

        public string Posicion { get; set; }

        public int Numero { get; set; }

        public string Condicion { get; set; }
    }
}
```

La clase contiene las propiedades necesarias para almacenar la información de cada jugador registrado.

### Paso 4: Declaración de listas

Dentro del formulario principal se creó una lista genérica para almacenar los jugadores en memoria.

```
CSharp
List<Jugador> listaJugadores = new List<Jugador>();
```

También se declaró una variable índice para seleccionar registros del DataGridView.

```
CSharp
int indice;
```

Esta lista permite gestionar la información sin utilizar base de datos.

### Paso 5: Implementación del método Mostrar()

Se creó un método encargado de actualizar automáticamente el DataGridView mostrando todos los jugadores registrados.

```
CSharp
public void Mostrar()
{
    dataGridView1.DataSource = null;
    dataGridView1.DataSource = listaJugadores;
}
```

El método se ejecuta cada vez que se agrega, modifica o elimina un jugador.

### Paso 6: Programación del botón Agregar

Se programó el botón Agregar con la finalidad de registrar nuevos jugadores dentro de la lista en memoria.

Para ello se creó un objeto de la clase Jugador y posteriormente se asignaron los datos ingresados desde los controles del formulario.

```
CSharp
private void btnAgregar_Click(object sender, EventArgs e)
{
    Jugador j = new Jugador();

    j.Nombre = txtNombre.Text;
    j.Posicion = txtPosicion.Text;
    j.Numero = int.Parse(txtNumero.Text);
    j.Condicion = cbCondicion.Text;

    listaJugadores.Add(j);

    Mostrar();

    MessageBox.Show("Jugador agregado");
}
```

El sistema realiza las siguientes acciones:

- Captura la información ingresada.
- Crea un objeto tipo Jugador.
- Guarda el objeto en la lista.
- Actualiza automáticamente el DataGridView.
- Muestra un mensaje de confirmación.

### **Paso 7: Programación del botón Eliminar**

Se implementó el botón Eliminar para remover jugadores registrados de la lista.

```
CSharp
private void btnEliminar_Click(object sender, EventArgs e)
{
    listaJugadores.RemoveAt(indice);

    Mostrar();

    MessageBox.Show("Jugador eliminado");
}
```

El sistema elimina el jugador seleccionado utilizando el índice obtenido desde el DataGridView.

### **Paso 8: Programación del botón Modificar**

Se programó el botón Modificar con el objetivo de actualizar la información de un jugador previamente registrado.

```
CSharp
private void btnModificar_Click(object sender, EventArgs e)
{
    listaJugadores[indice].Nombre = txtNombre.Text;
    listaJugadores[indice].Posicion = txtPosicion.Text;
    listaJugadores[indice].Numero = int.Parse(txtNumero.Text);
    listaJugadores[indice].Condicion = cbCondicion.Text;

    Mostrar();

    MessageBox.Show("Jugador modificado");
}
```

El sistema actualiza los datos del jugador seleccionado y refresca automáticamente el DataGridView.

### **Paso 9: Programación del DataGridView**

Se implementó el evento CellClick del DataGridView para permitir seleccionar registros y mostrar automáticamente la información en los controles del formulario.

```
CSharp
private void dataGridView1_CellClick(object sender, DataGridViewCellEventArgs e)
```

```
{  
    indice = e.RowIndex;  
  
    txtNombre.Text = listaJugadores[indice].Nombre;  
    txtPosicion.Text = listaJugadores[indice].Posicion;  
    txtNumero.Text = listaJugadores[indice].Numero.ToString();  
    cbCondicion.Text = listaJugadores[indice].Condicion;  
}
```

Este procedimiento permitió facilitar la modificación y eliminación de registros.

### **Paso 10: Ejecución y pruebas**

Finalmente se ejecutó la aplicación para verificar el correcto funcionamiento del sistema.

Durante las pruebas se realizaron las siguientes acciones:

- Registro de jugadores.
- Modificación de información.
- Eliminación de registros.
- Visualización de datos en el DataGridView.

Los resultados obtenidos fueron satisfactorios, permitiendo gestionar correctamente la alineación de jugadores mediante Programación Orientada a Objetos y listas en memoria.



## IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

Interfaz principal del sistema:



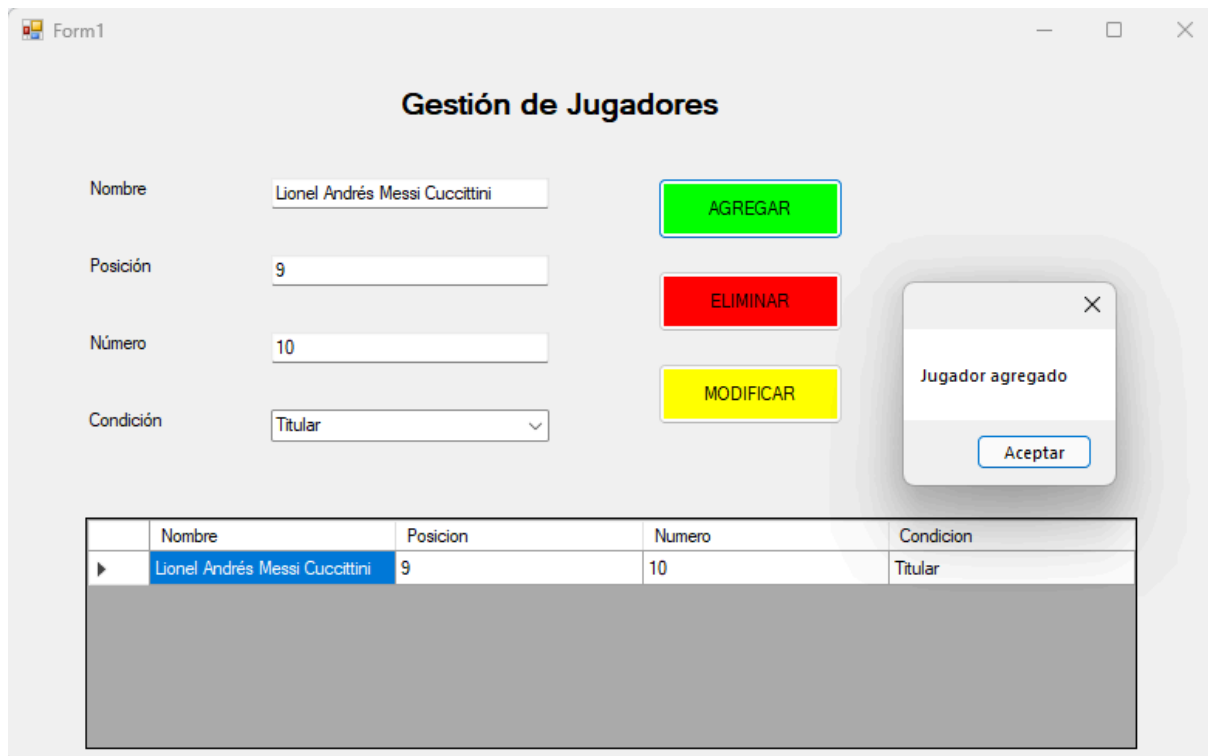
The screenshot shows a Windows Form titled 'Form1' with a light gray background. The title bar includes standard Windows window controls (minimize, maximize, close). The main content area is titled 'Gestión de Jugadores' in bold black text. Below the title, there are four input fields on the left, each with a label: 'Nombre', 'Posición', 'Número', and 'Condición'. The 'Condición' field is a dropdown menu. To the right of these fields are three action buttons: a green 'AGREGAR' button, a red 'ELIMINAR' button, and a yellow 'MODIFICAR' button. At the bottom of the form, there is a large, empty gray rectangular area, likely intended for a DataGridView to display player data.

**Análisis:** La interfaz gráfica desarrollada en Windows Forms permite registrar y administrar jugadores de manera sencilla y ordenada.

Los controles utilizados facilitaron la interacción del usuario con el sistema, permitiendo ingresar información mediante TextBox, ComboBox y botones de acción.

Además, el diseño permitió visualizar los datos en tiempo real utilizando el DataGridView.

## Registro de jugadores:



| Nombre                         | Posicion | Numero | Condicion |
|--------------------------------|----------|--------|-----------|
| Lionel Andrés Messi Cuccittini | 9        | 10     | Titular   |

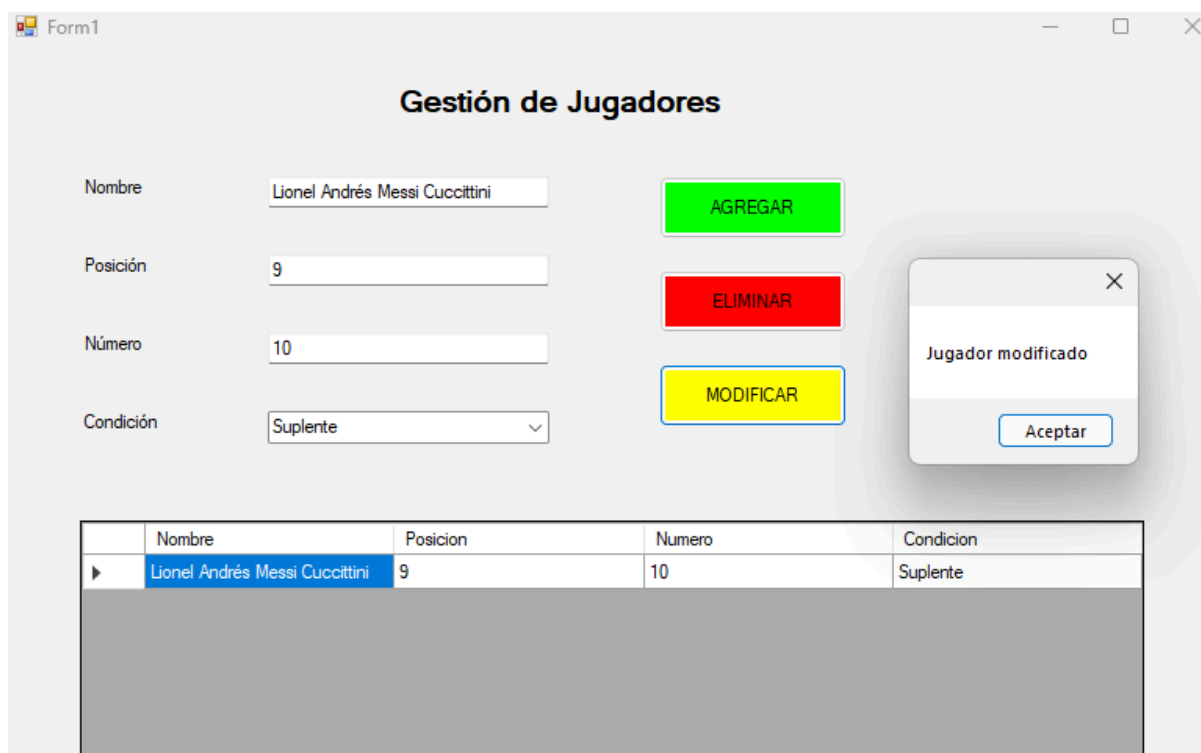
**Análisis:** El sistema registró correctamente los datos ingresados por el usuario, creando objetos de la clase Jugador y alimentándose en una lista en memoria.

Cada vez que se agregaba un jugador:

- la lista se actualizaba,
- el DataGridView mostraba automáticamente la información,
- y el sistema confirmaba la operación mediante mensajes.

Esto permitió verificar el correcto funcionamiento del botón Agregar.

### Modificación de jugadores:



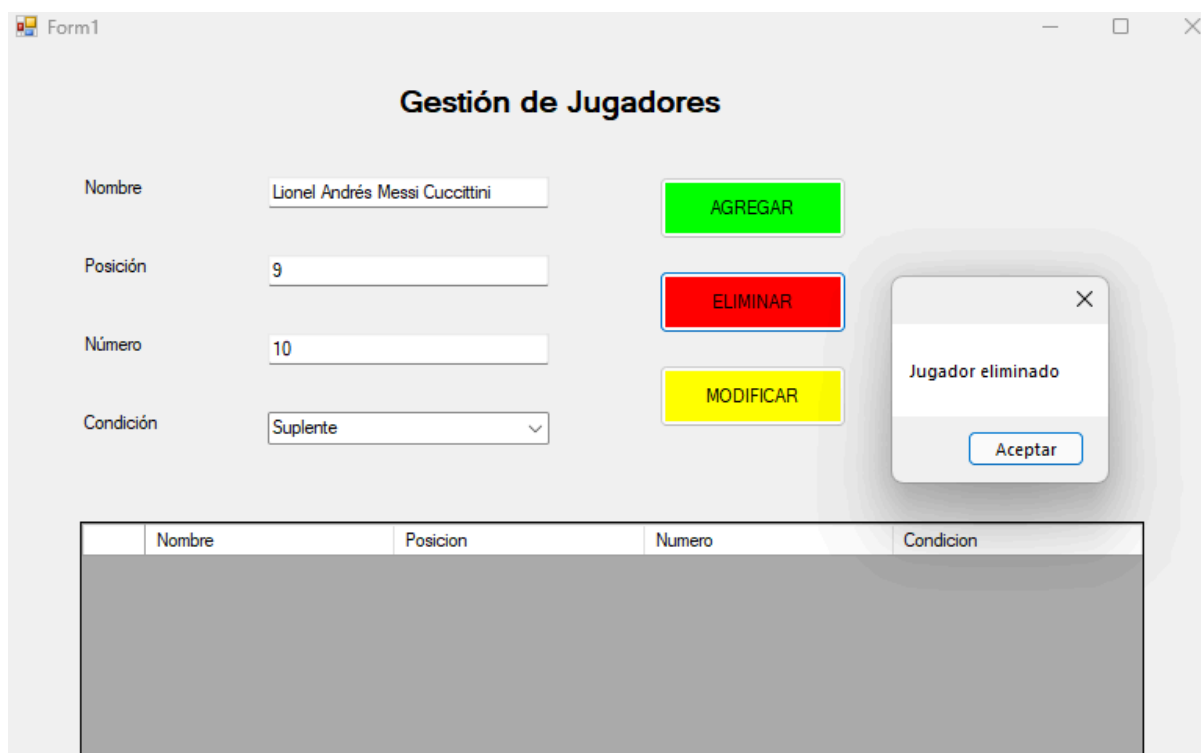
| Nombre                         | Posicion | Numero | Condicion |
|--------------------------------|----------|--------|-----------|
| Lionel Andrés Messi Cuccittini | 9        | 10     | Suplente  |

**Análisis:** La aplicación permite modificar correctamente los datos del jugador seleccionado desde el DataGridView.

Al seleccionar una fila:

- la información se cargaba automáticamente en los controles,
- el usuario podía editar los datos,
- y posteriormente actualizar la información mediante el botón Modificar.

Los cambios se reflejan inmediatamente en el DataGridView.

**Eliminación de jugadores:**

| Nombre | Posicion | Numero | Condicion |
|--------|----------|--------|-----------|
|--------|----------|--------|-----------|

**Análisis:** El sistema eliminó correctamente los jugadores seleccionados de la lista en memoria.

La operación se realizó utilizando el índice del registro seleccionado en el DataGridView, actualizando automáticamente la tabla visual luego de eliminar el elemento.

Esto permitió mantener la información organizada y actualizada.

**Funcionamiento general del sistema:**

El sistema desarrollado cumplió correctamente con los requisitos solicitados en el laboratorio. Se logró implementar:

- Programación Orientada a Objetos,
- manejo de listas genéricas,
- eventos de Windows Forms,
- y operaciones CRUD sin utilizar base de datos.

El funcionamiento general fue satisfactorio y permitió administrar correctamente la alineación de jugadores.

## **V. CUESTIONARIO**

### **1. ¿Qué es Windows Forms?**

Windows Forms es una tecnología de Microsoft utilizada para desarrollar aplicaciones de escritorio con interfaz gráfica en C#.

### **2. ¿Qué es Programación Orientada a Objetos?**

Es un paradigma de programación basado en clases y objetos que permite organizar y reutilizar código.

### **3. ¿Qué es una clase?**

Es una plantilla que define propiedades y comportamientos para crear objetos.

### **4. ¿Qué es List?**

Es una colección genérica utilizada para almacenar objetos dinámicamente en memoria.

### **5. ¿Qué función cumple el DataGridView?**

Permite mostrar información en forma de tabla dentro de Windows Forms.

## **CONCLUSIONES**

- Se desarrolló correctamente una aplicación Windows Forms.
- Se aplicaron conceptos de Programación Orientada a Objetos.
- Se utilizaron listas genéricas para almacenar información.
- Se implementaron operaciones CRUD correctamente.
- El DataGridView permitió visualizar los registros de manera ordenada.

## **RECOMENDACIONES**

- Validar campos vacíos.
- Evitar números repetidos.
- Mejorar el diseño visual.
- Implementar almacenamiento en base de datos en futuras versiones.

## **BIBLIOGRAFÍA**

- Deitel, P. C#. Cómo programar. Pearson Educación.



- Microsoft Corporation. Manual de programación en C#.
- Joyanes Aguilar, L. Fundamentos de Programación.

## **WEBGRAFÍA**

- Microsoft Learn. Documentación oficial de C#.
- Microsoft Learn. Guía de LINQ en .NET.
- Microsoft Learn. Windows Forms Documentation.